

Quicksort

Deterministic
Randomized

References:

Introduction to Algorithms

by Cormen, Leiserson, Rivest & Stein [Chapter 7]

Fundamentals of Algorithmics

by Gilles Brassard and Paul Bratley [Chapter 10]

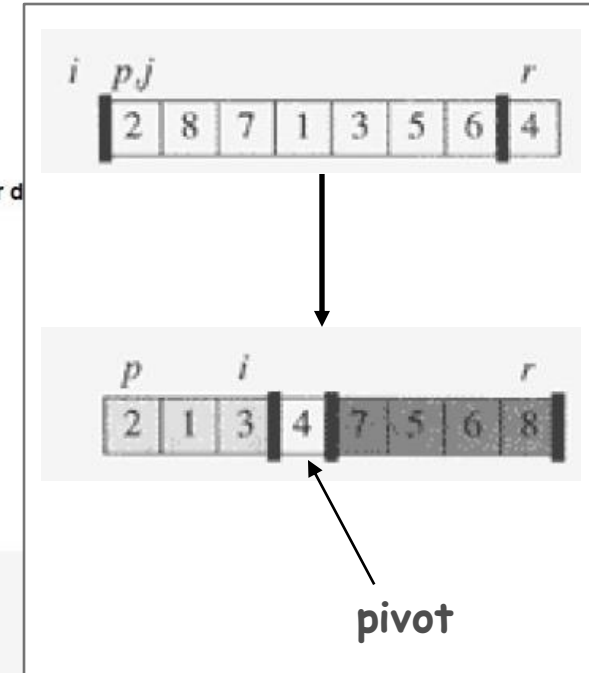
Quicksort

- Divide and conquer paradigm
- In place sorting
- Fast in practice
 - Worst case: $\Theta(n^2)$
- Randomized quicksort
 - Expected running time: $O(n \log n)$

Divide and Conquer

Copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or distribution.

A: array to be *in-place* sorted.
p and *r* are the start and the end indices.
q is the final index of the pivot.



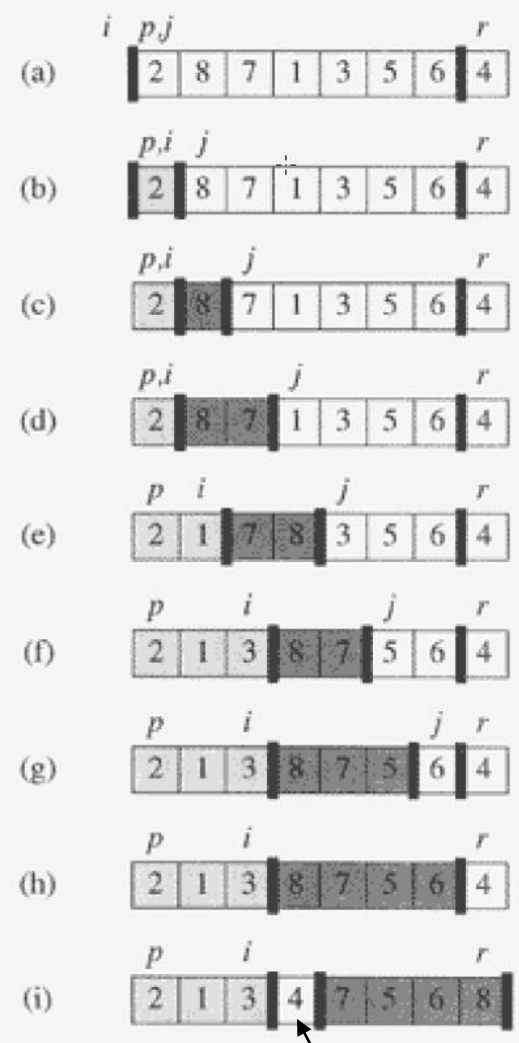
QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2      then  $q \leftarrow \text{PARTITION}(A, p, r)$ 
3          QUICKSORT( $A, p, q - 1$ )
4          QUICKSORT( $A, q + 1, r$ )
```

PARTITION(A, p, r)

```

1   $x \leftarrow A[r]$ 
2   $i \leftarrow p - 1$ 
3  for  $j \leftarrow p$  to  $r - 1$ 
4      do if  $A[j] \leq x$ 
5          then  $i \leftarrow i + 1$ 
6              exchange  $A[i] \leftrightarrow A[j]$ 
7  exchange  $A[i + 1] \leftrightarrow A[r]$ 
8  return  $i + 1$ 
    
```



pivot

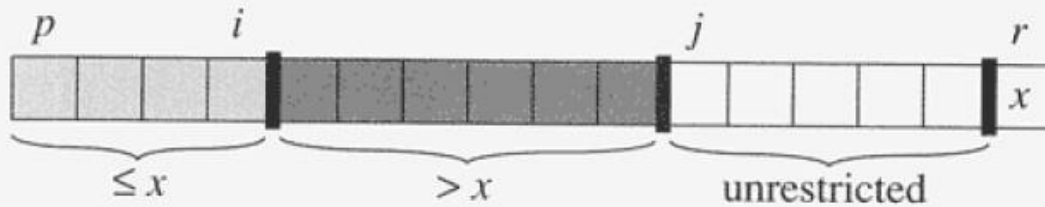


Figure 7.2 The four regions maintained by the procedure PARTITION on a subarray $A[p..r]$. The values in $A[p..i]$ are all less than or equal to x , the values in $A[i+1..j-1]$ are all greater than x , and $A[r] = x$. The values in $A[j..r-1]$ can take on any values.

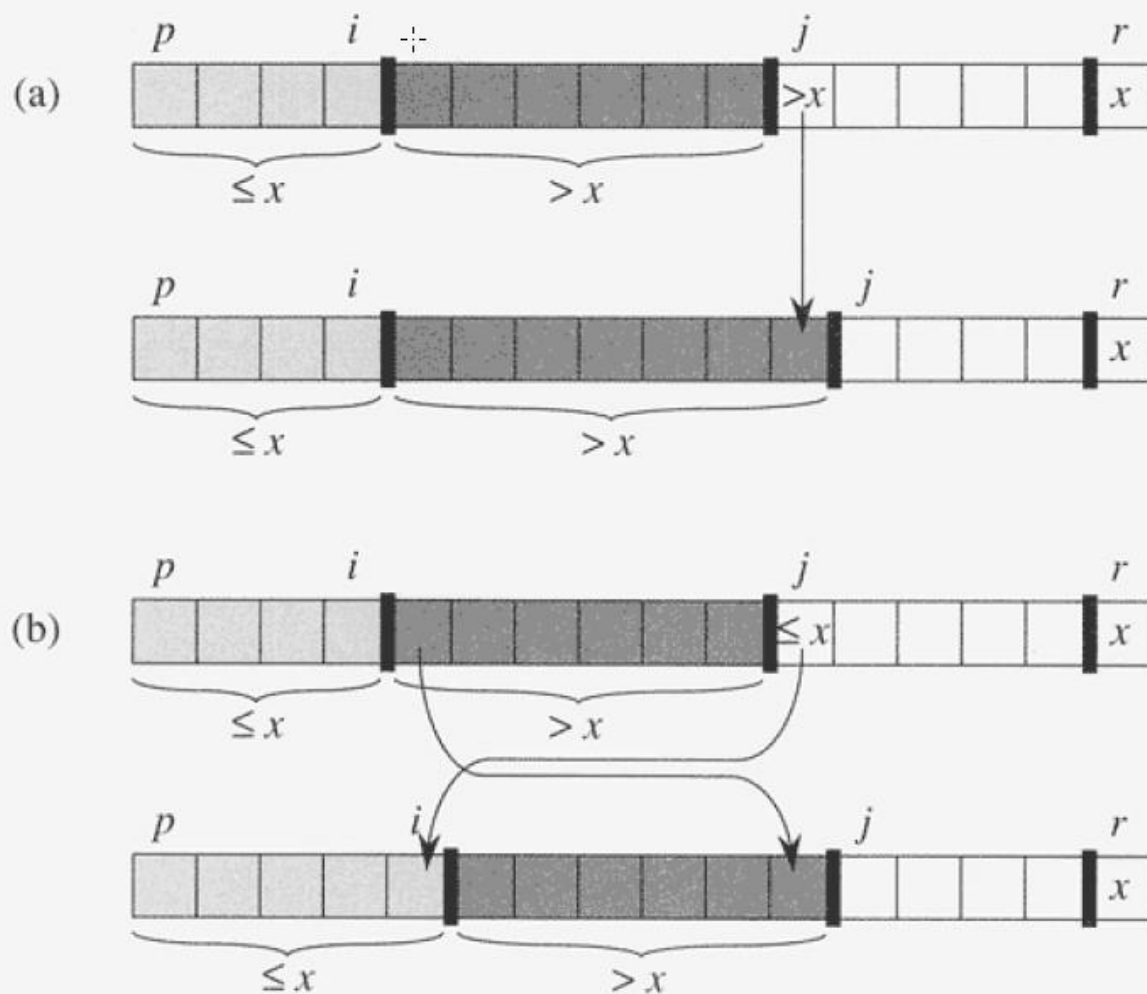


Figure 7.3 The two cases for one iteration of procedure PARTITION. (a) If $A[j] > x$, the only action is to increment j , which maintains the loop invariant. (b) If $A[j] \leq x$, index i is incremented, $A[i]$ and $A[j]$ are swapped, and then j is incremented. Again, the loop invariant is maintained.

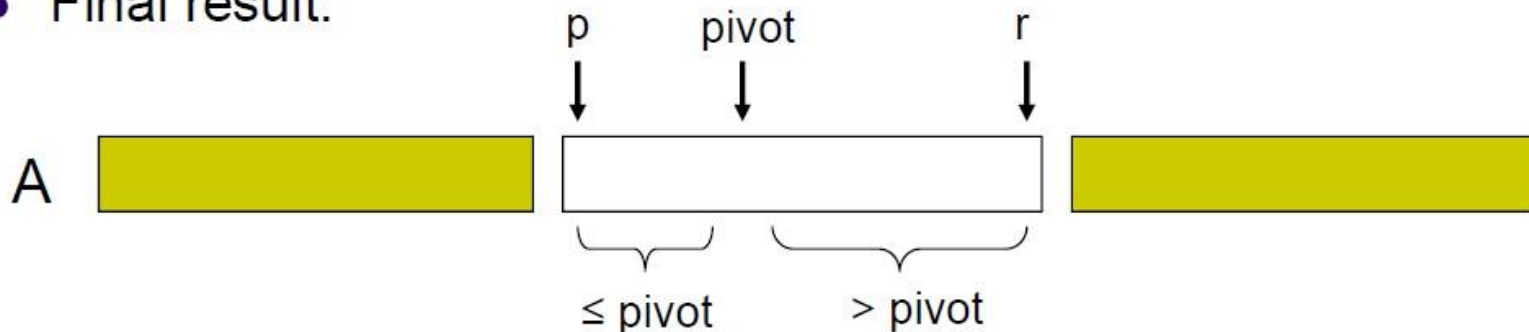


Trace of Partition

$\text{PARTITION}(A, p, r)$
 1 $i \leftarrow p - 1$
 2 **for** $j \leftarrow p$ **to** $r - 1$
 3 **if** $A[j] \leq A[r]$
 4 $i \leftarrow i + 1$
 5 swap $A[i]$ and $A[j]$
 6 swap $A[i + 1]$ and $A[r]$
 7 **return** $i + 1$

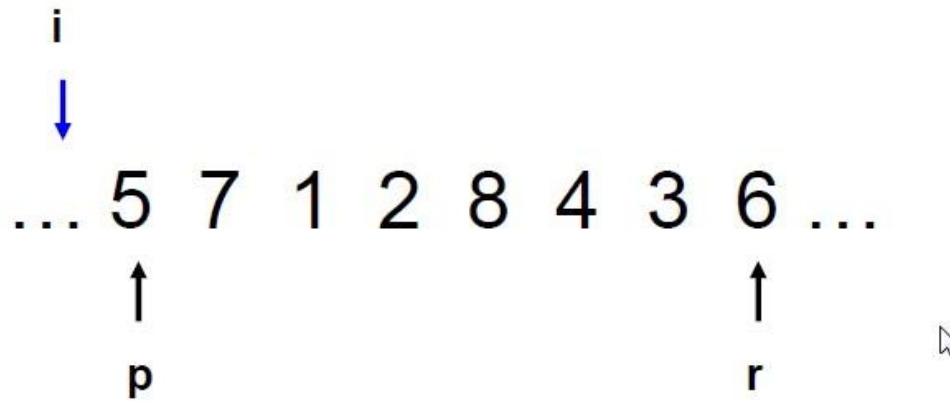


- $A[r]$ is called the **pivot**
- Partitions the elements $A[p \dots r-1]$ into two sets, those $\leq$ pivot and those $>$ pivot
- Operates in place
- Final result:




$$\text{PARTITION}(A, p, r)$$

10



PARTITION(A, p, r)

```
1   $i \leftarrow p - 1$ 
2  for  $j \leftarrow p$  to  $r - 1$ 
3      if  $A[j] \leq A[r]$ 
4           $i \leftarrow i + 1$ 
5          swap  $A[i]$  and  $A[j]$ 
6  swap  $A[i + 1]$  and  $A[r]$ 
7  return  $i + 1$ 
```


$$1 \quad i \leftarrow p - 1$$
3 **if** $A[j] \leq A[r]$

```
5 swap A[i] and A[j]
```

```

7  return  $i + 1$ 

```



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

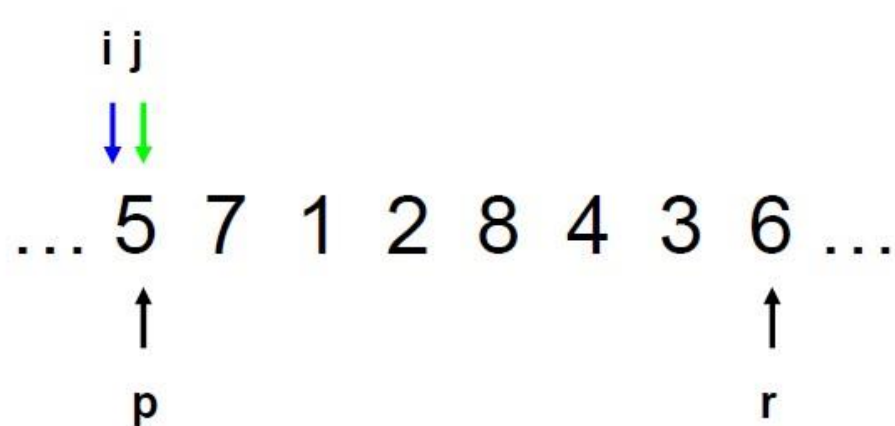
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

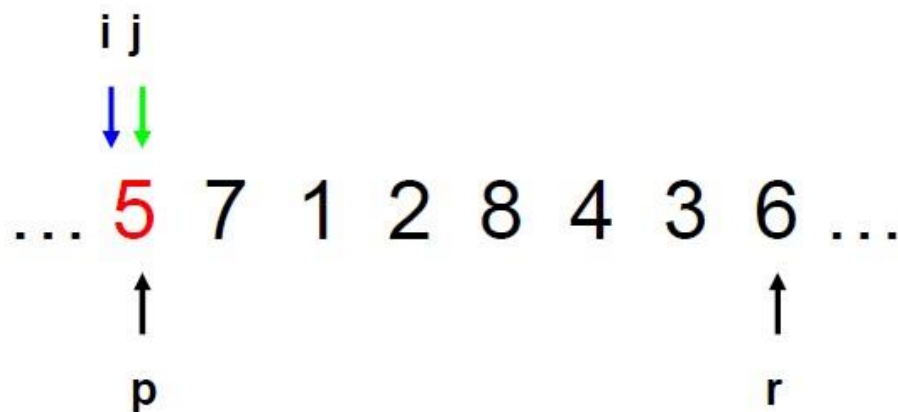
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

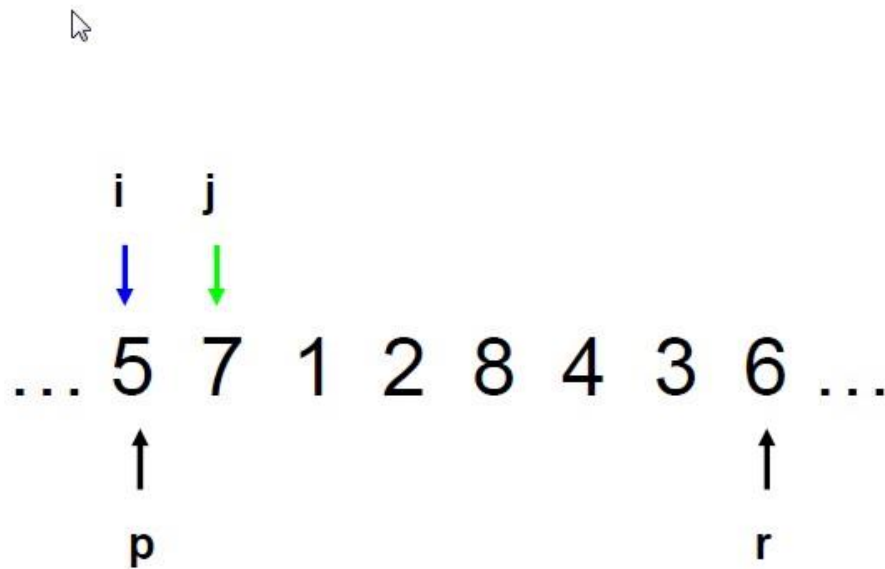
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

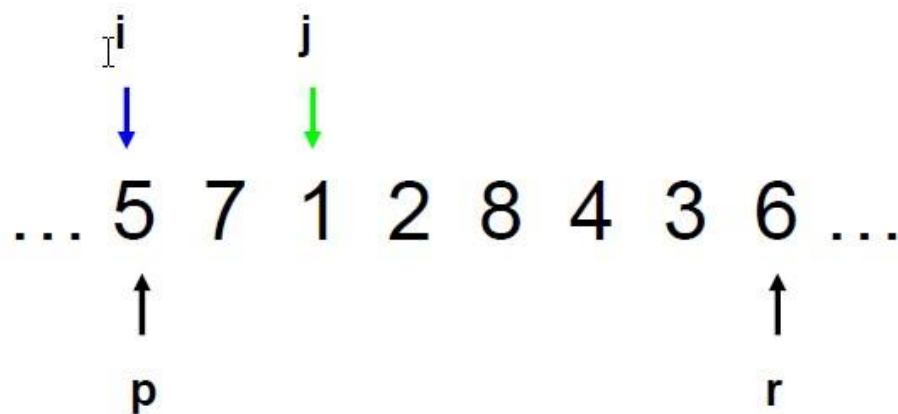
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

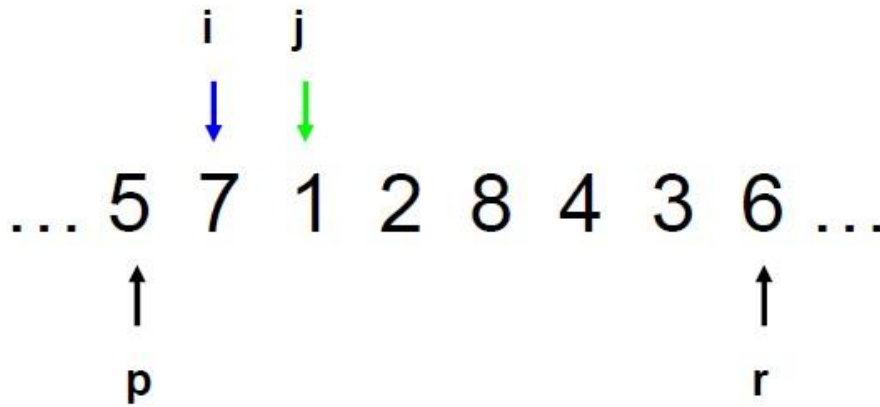
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

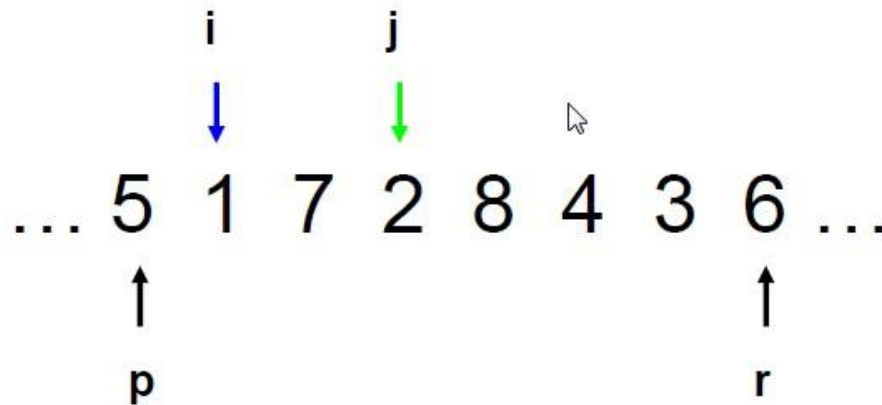
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 **swap** $A[i]$ and $A[j]$

6 **swap** $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

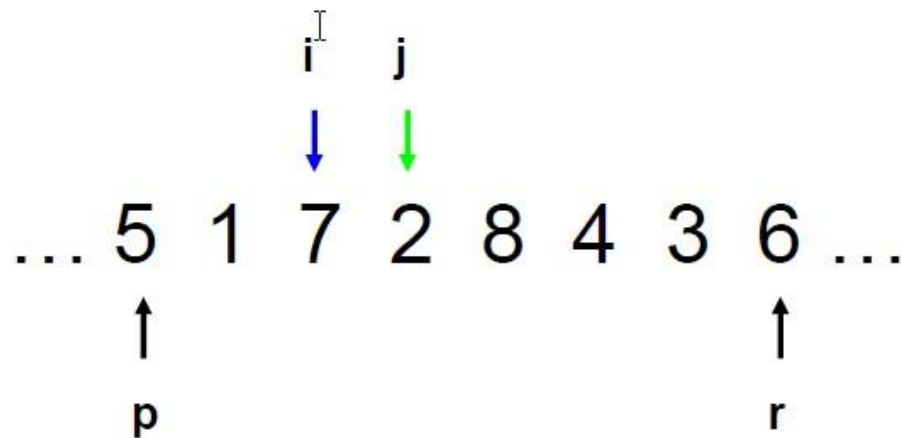
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

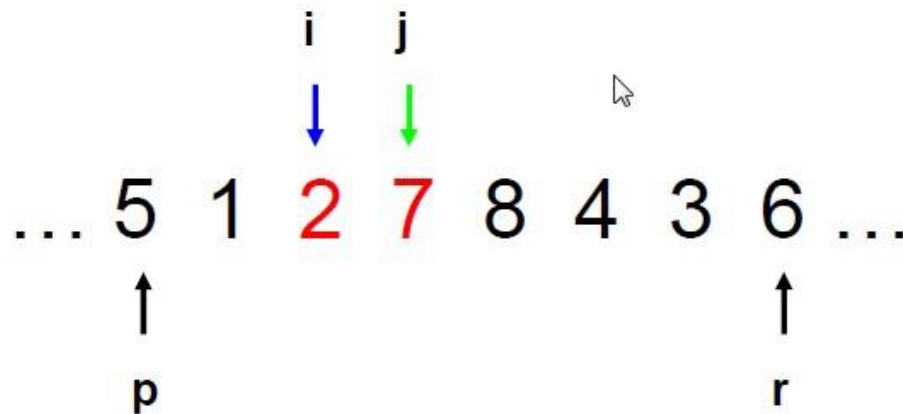
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 **swap** $A[i]$ and $A[j]$

6 **swap** $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

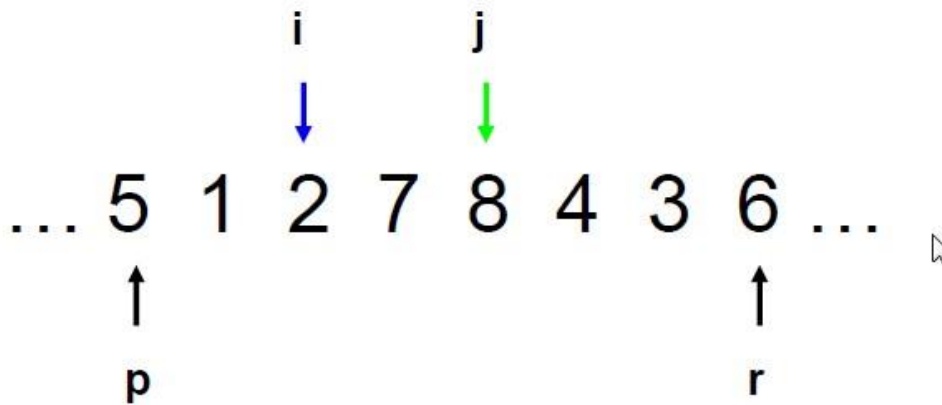
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

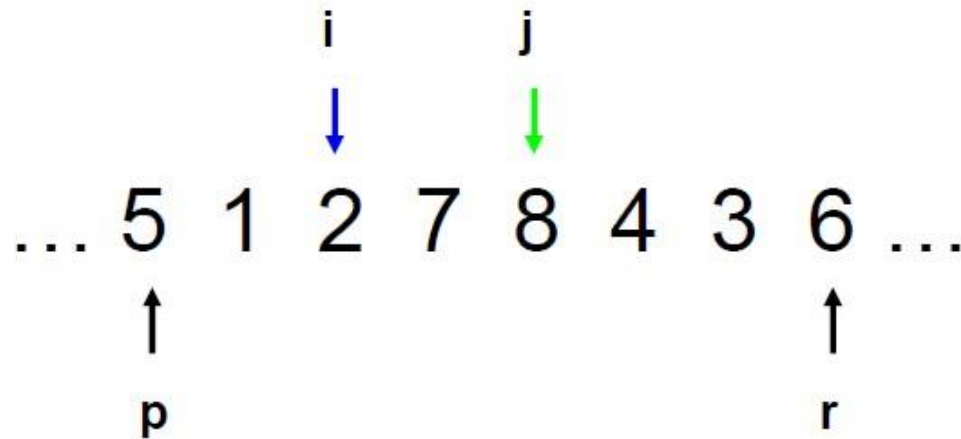
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

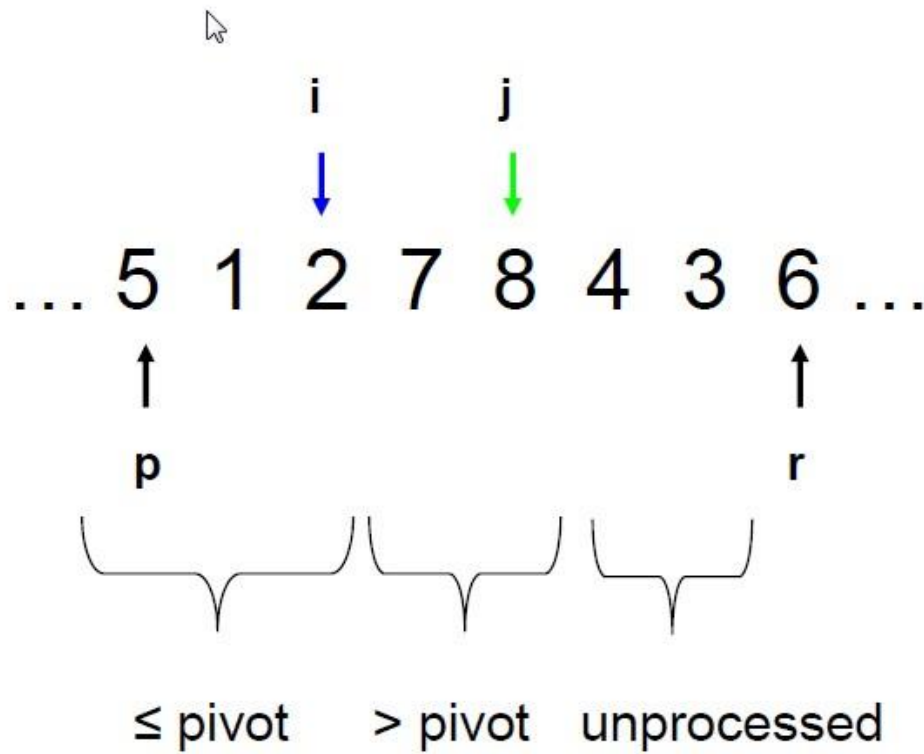
5 swap $A[i]$ and $A[j]$

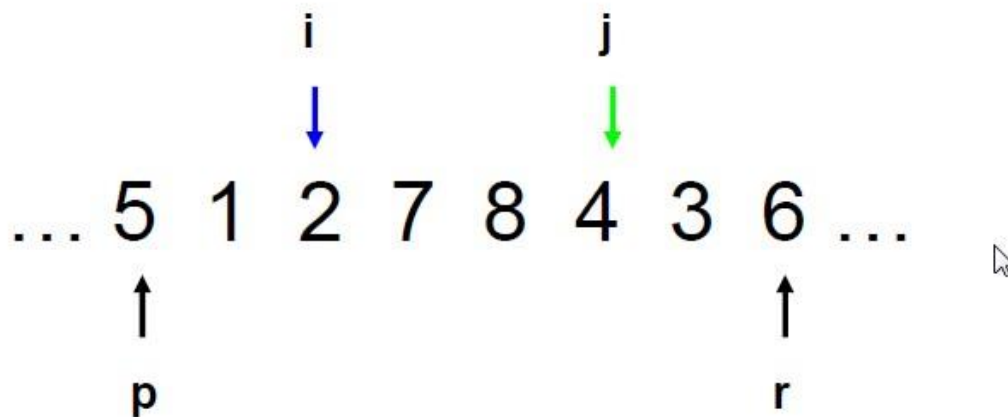
6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



What's happening?





PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

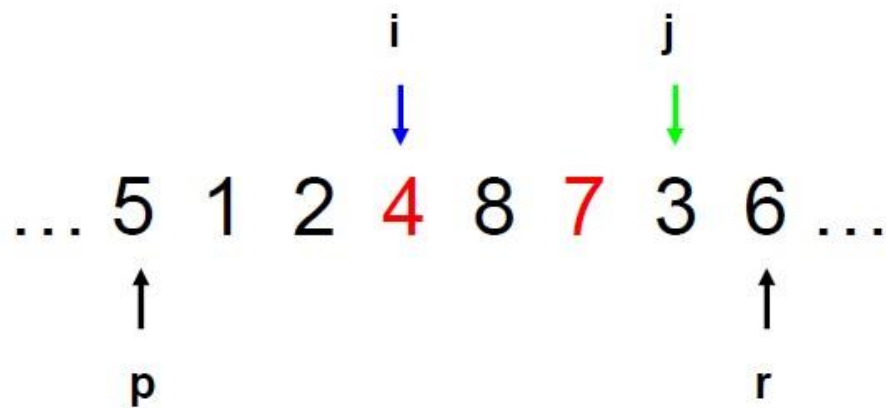
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

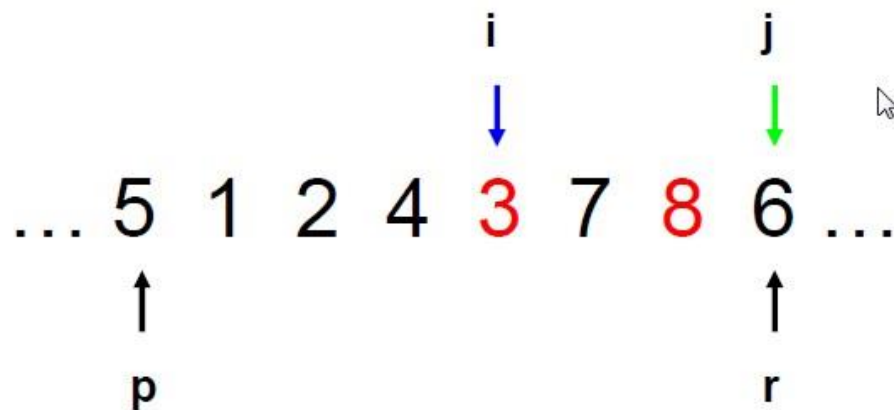
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

1 $i \leftarrow p - 1$

2 **for** $j \leftarrow p$ **to** $r - 1$

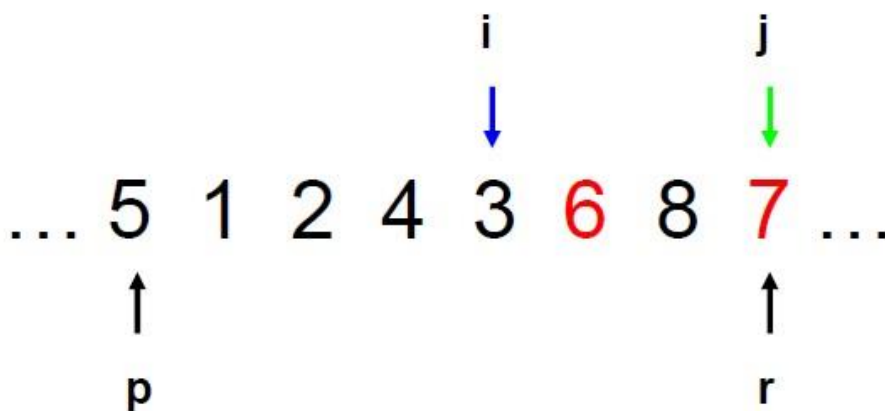
3 **if** $A[j] \leq A[r]$

4 $i \leftarrow i + 1$

5 swap $A[i]$ and $A[j]$

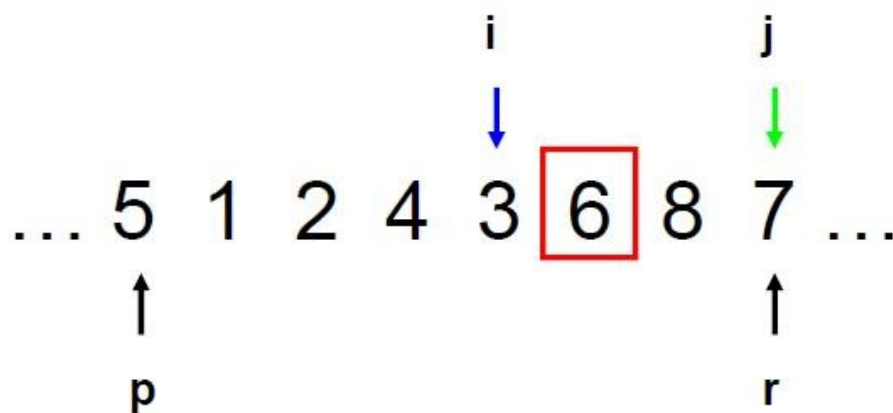
6 swap $A[i + 1]$ and $A[r]$

7 **return** $i + 1$



PARTITION(A, p, r)

```
1   $i \leftarrow p - 1$ 
2  for  $j \leftarrow p$  to  $r - 1$ 
3      if  $A[j] \leq A[r]$ 
4           $i \leftarrow i + 1$ 
5          swap  $A[i]$  and  $A[j]$ 
6  swap  $A[i + 1]$  and  $A[r]$ 
7  return  $i + 1$ 
```



PARTITION(A, p, r)

```
1   $i \leftarrow p - 1$ 
2  for  $j \leftarrow p$  to  $r - 1$ 
3      if  $A[j] \leq A[r]$ 
4           $i \leftarrow i + 1$ 
5          swap  $A[i]$  and  $A[j]$ 
6  swap  $A[i + 1]$  and  $A[r]$ 
7  return  $i + 1$ 
```



Trace of Quicksort

Quicksort



QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q \leftarrow \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

PARTITION(A, p, r)

```
1   $i \leftarrow p - 1$ 
2  for  $j \leftarrow p$  to  $r - 1$ 
3      if  $A[j] \leq A[r]$ 
4           $i \leftarrow i + 1$ 
5          swap  $A[i]$  and  $A[j]$ 
6  swap  $A[i + 1]$  and  $A[r]$ 
7  return  $i + 1$ 
```



8 5 1 3 6 2 7 4

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



8 5 1 3 6 2 7 4

QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 3 2 4 6 8 7 5

QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 3 2 4 6 8 7 5

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 3 2 4 6 8 7 5

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 6 8 7 5

QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 6 8 7 5

QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 6 8 7 5

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 5 8 7 6

What happens here?

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 5 8 7 6

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 5 8 7 6

QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 5 6 7 8

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



1 2 3 4 5 6 7 8

QUICKSORT(A, p, r)

1 if $p < r$

2 $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)



Quicksort Analysis

- (1) “Bad” split on sorted lists
- (2) “Bad” split vs Uneven split
- (3) Average case analysis

Sorted List : Partition (Beginning and ending of each iteration)

I



i	p j			r
	1	2	3	4

	p i j			r
	1	2	3	4

	p i	j		r
	1	2	3	4

	p	ij		r
	1	2	3	4

	p	i	j	r
	1	2	3	4

	p		ij	r
	1	2	3	4

	p		ij	r
	1	2	3	4



Red: $O(n)$ comparisons and $O(n)$ (trivial) swaps for each call of Partition;
 $T(n) = T(n-1) + O(n)$;
 $O(n^2)$ swaps for quicksort

Reverse Sorted List : **Partition** (Beginning and ending of each iteration)

I
↓

i	p j			r
	4	3	2	1

	p i j			r
	4	3	2	1

	p i	j		r
	4	3	2	1

	p i	j		r
	4	3	2	1

	p i		j	r
	4	3	2	1

	p i		j	r
	4	3	2	1

	p i		j	r
	1	3	2	4

↑

Red: $O(n)$ comparisons and one swap for each call of **Partition**;
 $T(n) = T(n-1) + O(n)$
 $O(n)$ swaps for quicksort
 $O(n^2)$ comparisons for quicksort

^IQuicksort Average case?

- How close to “even” split do we need to maintain an $O(n \log n)$ running time?
 - Say the Partition procedure always splits the array into some constant ratio b -to- a , e.g., 9-to-1
 - What is the recurrence?

$$T(n) \leq T\left(\frac{a}{a+b}n\right) + T\left(\frac{b}{a+b}n\right) + cn$$

$$E.g., T(n) \leq T\left(\frac{9}{10}n\right) + T\left(\frac{1}{10}n\right) + cn$$

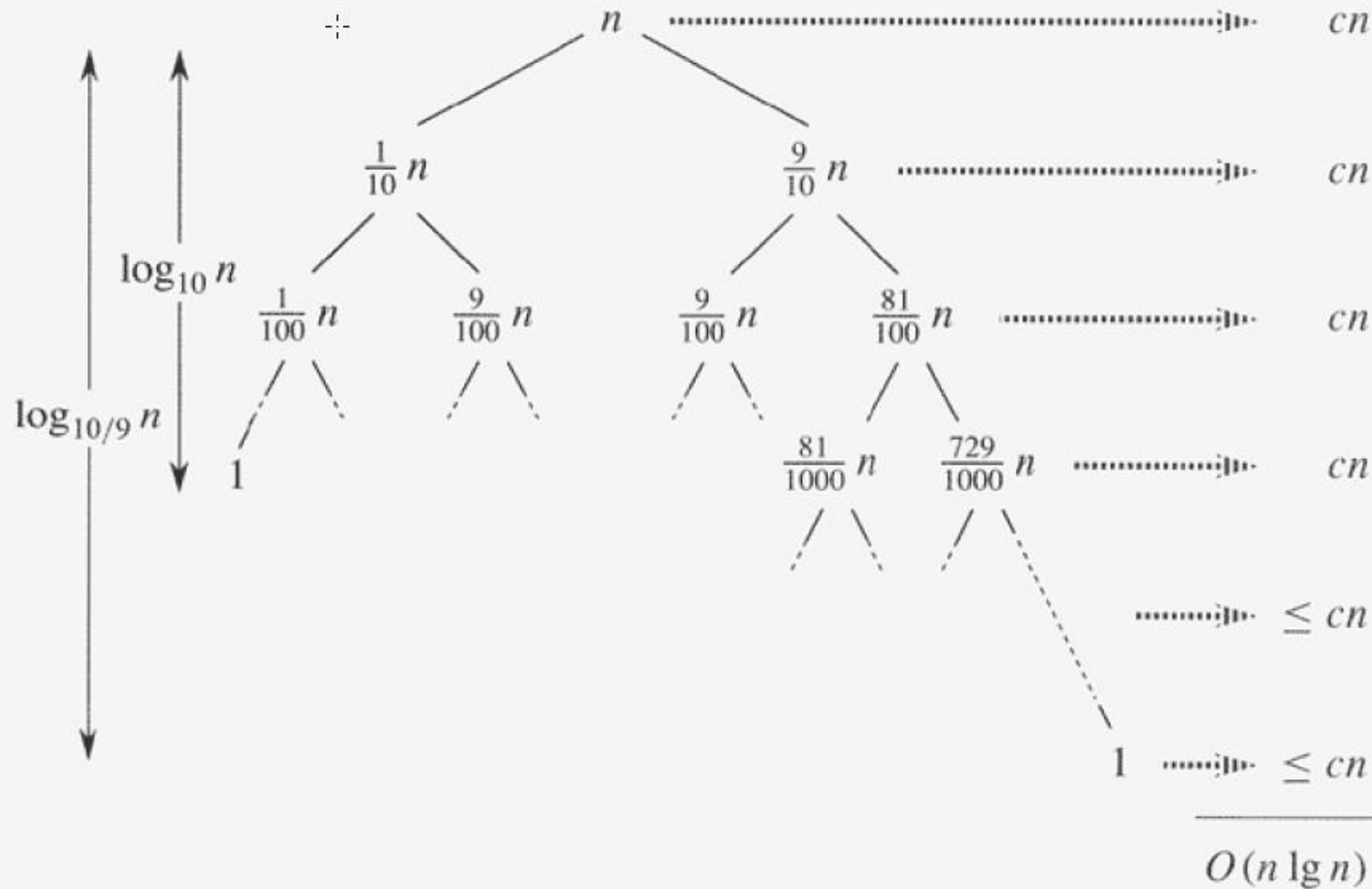
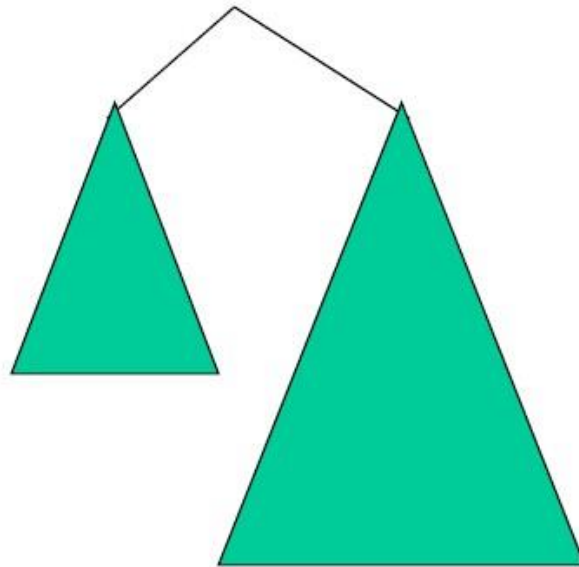


Figure 7.4 A recursion tree for QUICKSORT in which PARTITION always produces a 9-to-1 split, yielding a running time of $O(n \lg n)$. Nodes show subproblem sizes, with per-level costs on the right. The per-level costs include the constant c implicit in the $\Theta(n)$ term.

I What is the depth of the tree?

- Leaves will have different heights
- Want to pick the deepest leaf
- Assume $a < b$



What is the depth of the tree?

- Assume $a < b$

$$\left(\frac{b}{a+b}\right)^d n = 1$$

...

$$d = \log_{\frac{a+b}{b}} n$$

Mixing “Good” and “Bad” Splits

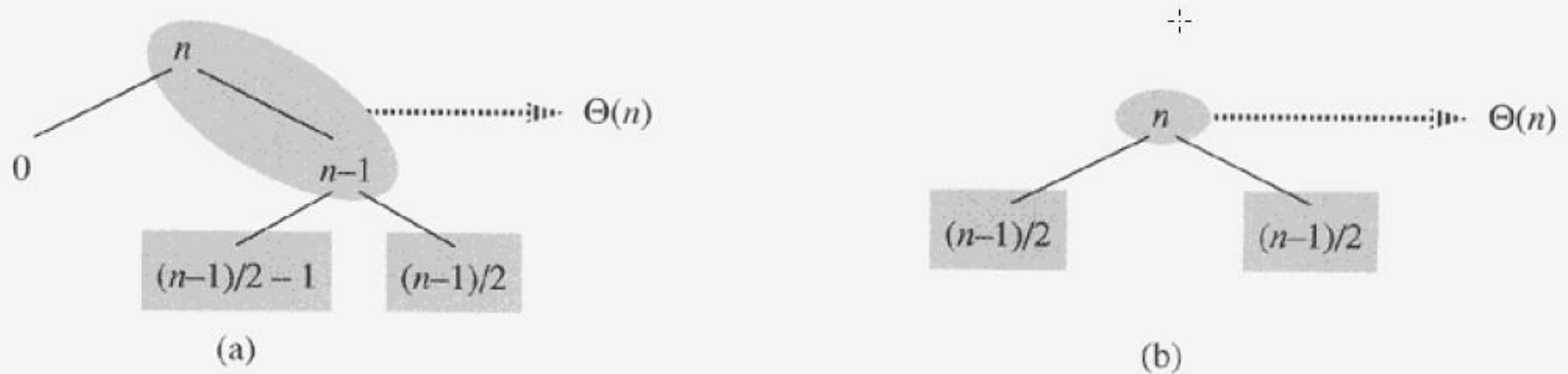
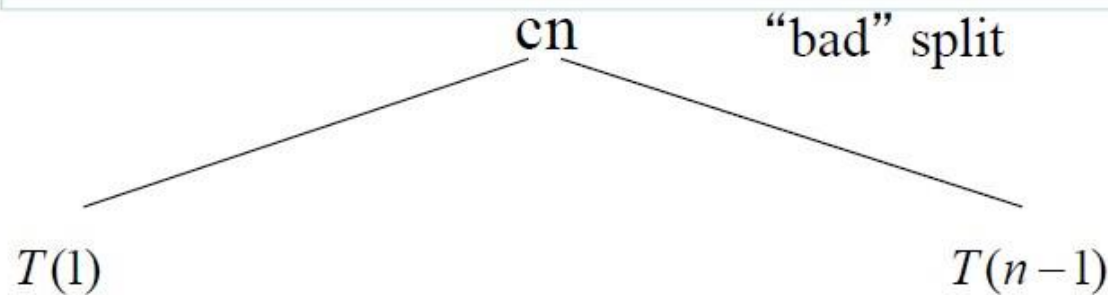


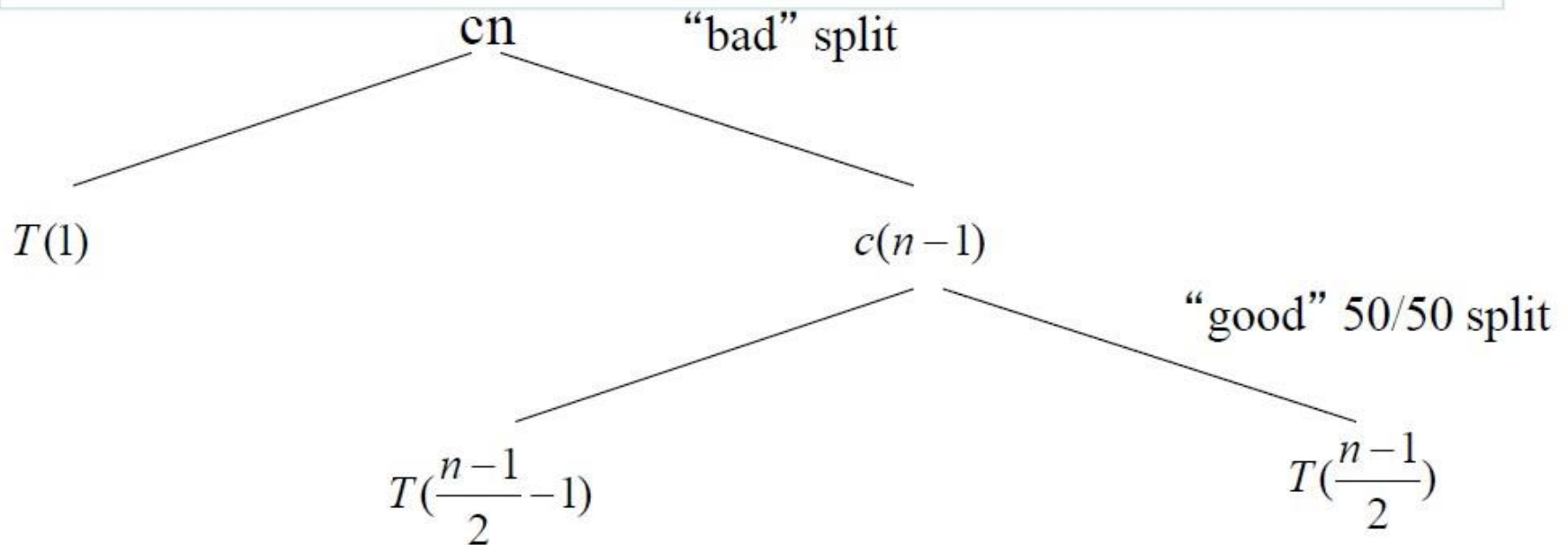
Figure 7.5 (a) Two levels of a recursion tree for quicksort. The partitioning at the root costs n and produces a “bad” split: two subarrays of sizes 0 and $n - 1$. The partitioning of the subarray of size $n - 1$ costs $n - 1$ and produces a “good” split: subarrays of size $(n - 1)/2 - 1$ and $(n - 1)/2$. (b) A single level of a recursion tree that is very well balanced. In both parts, the partitioning cost for the subproblems shown with elliptical shading is $\Theta(n)$. Yet the subproblems remaining to be solved in (a), shown with square shading, are no larger than the corresponding subproblems remaining to be solved in (b).

Quicksort average case



Limited number of “bad splits” followed by a “good split” is a slower “good split”!!

Quicksort average case :
 “bad split” + “good split” $\Rightarrow$ “good split”



$$T(n) = \underbrace{T(1) + T(\frac{n-1}{2}-1) + T(\frac{n-1}{2})}_{\text{recursion cost}} + \underbrace{\Theta(n) + \Theta(n-1)}_{\text{partition cost}}$$

Average Case Behavior: Probabilistic Analysis
vs. Average Case Design: Randomized Algorithms

- Probabilistic analysis of a deterministic algorithm:
 - assume probability distribution on the inputs
- Randomized algorithm design:
 - use random choices in the algorithm
 - E.g., generate random permutation of the input
 - E.g., pick a pivot randomly at each step

Randomized Algorithms

I

- Randomized algorithms make use of a randomizer (such as a random number generator).
- Some of the decisions of the algorithm depend on the output of the randomizer.
 - The output may vary from run to run.
 - The execution time may vary from run to run.

Two Major Classes of Randomized Algorithms

- Las Vegas algorithms:
 - Always produce the same (correct) output for the same input.
 - Execution time depends on the randomizer.
 - Goal is to minimize the odds of encountering worst case performance.
 - *Example:* Randomized quick sort

Two Major Classes of Randomized Algorithms

- Monte Carlo algorithms:
 - Give an incorrect answer with very low probability.
 - Typically does not display variation in execution time for a particular input.
 - Goal is to provide an answer that has a high likelihood of being correct in a reasonable time.
 - *Example:* Determining primality of an integer with several hundred decimal digits.

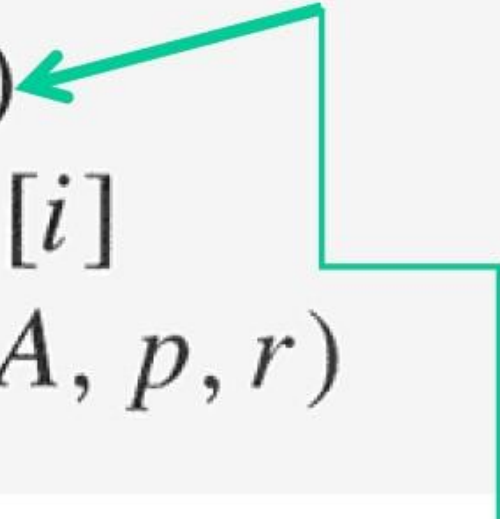


RANDOMIZED-PARTITION(A, p, r)

1 $i \leftarrow \text{RANDOM}(p, r)$

2 exchange $A[r] \leftrightarrow A[i]$

3 **return** PARTITION(A, p, r)



Pick pivot randomly

RANDOMIZED-QUICKSORT(A, p, r)

1 **if** $p < r$

2 **then** $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$

3 RANDOMIZED-QUICKSORT($A, p, q - 1$)

4 RANDOMIZED-QUICKSORT($A, q + 1, r$)

HOARE-PARTITION(A, p, r)

```
† 1   $x \leftarrow A[p]$ 
   2   $i \leftarrow p - 1$ 
   3   $j \leftarrow r + 1$ 
   4  while TRUE
   5      do repeat  $j \leftarrow j - 1$ 
   6          until  $A[j] \leq x$ 
   7      repeat  $i \leftarrow i + 1$ 
   8          until  $A[i] \geq x$ 
   9      if  $i < j$ 
  10          then exchange  $A[i] \leftrightarrow A[j]$ 
  11          else return  $j$ 
```

QUICKSORT'(A, p, r)

```
1  while  $p < r$   
2      do  $\triangleright$  Partition and sort left subarray.  
3       $q \leftarrow \text{PARTITION}(A, p, r)$   
4      QUICKSORT'(A, p,  $q - 1$ )  
5       $p \leftarrow q + 1$ 
```

QUICKSORT(A, p, r)

1 **if** $p < r$

2 **then** $q \leftarrow \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)

QUICKSORT'(A, p, r)

1 **while** $p < r$

2 **do** $\triangleright$ Partition and sort left subarray.

3 $q \leftarrow \text{PARTITION}(A, p, r)$

4 QUICKSORT'($A, p, q - 1$)

5 $p \leftarrow q + 1$